COMP 581: Introduction to Robotics

Spring 2026
Instructor: Daniel Szafir (daniel.szafir@cs.unc.edu)
Teaching Assistant: Thayer Hicks (thayerh@cs.unc.edu)

Lab 1: Your First Robot

Deadlines

- Lab pre-flection due **Monday, February 2 at 11:59pm**. To pass this part of the lab, please submit your answers in a group submission on GradeScope (see the Lab 1 Pre-flection assignment link on Canvas).
- Lab simulation start and physical robot checkout due by **Wednesday, February 4 at 11:59pm**. Before the due time, each team must meet with the Teaching Assistant to demo their code in the simulator and sign out a robot kit. The code in the simulator does ***not*** need to be complete by this deadline, but **must show some progress** (e.g., the robot moves forward after the button is hit, as we worked through in class). Teams should also submit their code by committing and pushing it to GitHub; see specific instructions below**.**
- Robot lab due **Wednesday, February 18 at the beginning of class (11:15 am)**. Before the due time, each team should submit their final code for both their physical robot and the simulator by committing and pushing to GitHub; see specific instructions below**.** For the late policy, see the course syllabus. Please include two files (one for the simulator code and one for the robot) or otherwise make it obvious to us how to run your simulator code easily vs which file your robot ran on the live lab test.
- Lab reflection due **Friday, February 20 at 11:59pm**. To pass this part of the lab, please submit your answers in a group submission on GradeScope (see the Lab 1 Reflection assignment link on Canvas).

Objectives

In this lab, you will program a simulated robot in Webots and build and program a physical mobile robot as well as familiarize yourselves with the motors and sensors at your disposal. You will program both your simulated and physical robot to accomplish a locomotion task and your general strategy should be to first test your code in simulation and then deploying your solution on the real platform. The objectives of this lab include:

- Gaining experience with Lego Mindstorms EV3 and the Ev3 MicroPython API.
- Building further familiarity with the Webots robotics simulator and practice going from simulation to real robotics deployment.
- Putting the theoretical knowledge of state machines, sensors, and kinematics from our

class lectures into practice to solve a real robotics challenge.

Hints and Suggestions

Getting your first robot working will be a significant academic challenge. We suggest:

- Reading this entire document and making sure you understand the goals of this lab before attempting anything.
- Starting early.
- Asking questions to clarify any misunderstandings. Come to office hours if you need additional help. Review the lecture slides for relevant formulas.
- Getting your code fully working in the simulator before trying to transfer code to your real robot.
- Breaking the problem down into simple, discrete components. Build your code and test it for each component before moving on to the next component. Think about edge cases that might arise. Structure your overall program flow as a state machine (see the Lab 1 pre-flection).

Task 1: Development Setup

Subtask 1.A – Visual Studio Code and EV3 MicroPython

1. If you have not already set up the Lego Mindstorms EV3 on your laptop, start by reading the Tutorial at: https://education.lego.com/v3/assets/blt293eea581807678a/bltb470b9ea6e38f8d4/5f8802fc4376310c19e33714/getting-started-with-micropython-v2_enus.pdf

2. Follow all the installation instructions described in Chapter 1 to install Visual Studio Code and the EV3 MicroPython libraries. We have already flashed the MicroSD cards for you (Step 1.3), so you can skip that part.

3. Once you have your physical robot kit, follow all the guides for creating and testing simple programs described in Chapter 2. Make sure the simple programs in Chapter 2 (creating a beep noise and running a motor) are working properly before trying anything more advanced.

For more information about EV3 MicroPython, including some start programs for basic robot movement, see:
https://pybricks.com/ev3-micropython/index.html

Subtask 1.B – Lab 1 Webots Simulation

We have created a Webots simulation library that will enable you to write EV3 MicroPython compatible code in Webots to control a simulated Lego robot in a world that mimics the Lab 1 environment. To download everything you need for this simulation and for deploying your program to the real robot:

1. If you do not already have a GitHub account, create one at https://github.com/
    - To work with GitHub repositories, you may wish to also install GitHub Desktop https://desktop.github.com/, which can make things a bit easier than using the command line tools.
    - If you have never used GitHub before, please familiarize yourself with it first by reading through this tutorial: https://docs.github.com/en/get-started/quickstart/hello-world. You can also find many more written and video tutorials online with simple internet searches.
2. Log into your GitHub account and then accept the GitHub Classroom assignment by clicking this link: https://classroom.github.com/a/eB24STZY
3. The link will take you to a page where you can select your name from a list to associate your GitHub account with your name. Find your name in the list of Identifiers and click on it. If your name is not in the list, please contact the TA or Instructor.
4. You will then be taken to a page to create or join your Lab team.

- If you are the first team member to accept the assignment, give your team a name in the textbox and click the +Create Team button. Then click the "Accept this Assignment" button on the next page. GitHub will then set up your repository. After a minute, it should tell you that your Team's assignment repository has been created and show you the repository link (if it doesn't show this link, try refreshing the page).
- If you are not the first team member to accept the assignment, find your team under the "Join an existing team" list and click the "Join" button. You will then be taken to a page that has the link for your team repository.

5. Navigate to your repository link. You will see there is quite a bit of relevant information and links in the repository README.md file (you can simply scroll down to see this information). The repository has the following folders:
    - **Controllers**: within this folder is another folder called main. Within the main folder is the `main.py` file that you will edit, which is the controller for the Webots simulated robot. This `main.py` file is where you should do all your programming for this Lab.
    - **Lego_build_guide**: within this folder is a guide and reference images for building your physical Lego robot (see Task 2 below). You are welcome to ignore this guide and build your robot however you want if you'd like, but make sure your build conforms to the standards outlined in Task 2.
    - **Protos**: this folder contains information needed for setting up and visualizing the simulated Webots robot. You don't need to do anything with this folder.
    - **Worlds**: this folder contains the Webots world file `lab1.wbt` that sets up the Lab 1 simulation. You don't need to do anything with this folder.
6. Follow the "Code Setup" tutorial in the README.md file which explains how to clone the repository, open the `main.py` file in Visual Studio Code, and open the Lab 1 world file in Webots.
7. Read the information under the "Lab 1" header in the README.md file to learn more about running the Webots simulation, a few edits you'll need to make to your code when transferring it from Webots to the physical Lego robot, editing the simulated Webots world, etc.
8. You should now be able to edit the `main.py` controller file in Visual Studio Code and see the updates to the robot's behavior when you run the Webots simulation.
    - Always remember to reset the Webots time back to zero.
    - Your job is now to write the main.py controller to solve the challenges described below in Task 3 and to build a physical Lego robot as described below in Task 2 on which you will deploy your solution.
    - Note: for the simulated robot in Webots, here are the relevant parameters you may need for your kinematics modeling:
        i. r (wheel radius) = 0.03175
        ii. L (distance between wheels) = 0.127

Task 2: Build Your Robot

Once you have made some progress on the simulator, you must meet with the TA to demonstrate your progress and sign out a Lego Robot Kit. You do **not** need to have fully solved Lab 1 in the simulator in order to sign out your robot, but do need to demonstrate some amount of progress (at minimum, your robot drives forward).

Once you sign out your robot kit, you must use it to build a robot that accomplishes the objectives described in Task 3. Please refer to the information within the GitHub repository for build instructions and examples (see the "lego_build_guide" folder). You might want to match the example build provided there and in the simulator, but it is ultimately up to you how you want to build your robot, and we may give some bonus points for creative designs. However you choose build your robot, the following constraints apply:

1. The robot should be designed to operate on a flat floor with low-pile carpet.
2. You may only use pieces from your Lego Mindstorms kit. If you find any pieces from your kit don't seem to be working (e.g., sensors that don't report values), let the instructor or TA know and we can replace individual parts.
3. You must be able to press the dark gray center button such that it does not disturb the robot's position or sensors.
4. The robot must stay as one piece at all times.
5. The robot's height must be under 40 cm and cannot substantially change during the evaluation.
6. There must be one distinct *measuring point* at the front of the robot that will start at the center of the starting line. More specifically, your robot will be placed on the ground behind a starting line drawn on the ground, and your chosen measuring point must be a point on your robot directly above the starting line, and no part of your robot is permitted to be in front of the starting line. Your robot should be designed to accomplish the task described in Task 3 autonomously without any human intervention, except for pushing the dark gray center button as required.

Task 3: Odometry, Ranging, and Bumping

In this lab, your robot must demonstrate the capacity for autonomous locomotion. There are two specific subtasks that your robot must complete, described below. Note that both subtasks in this lab are to be completed autonomously, one after the other, in a single run, without picking up your robot. The only human intervention allowed is to push the center dark gray button as described below.

Pre-flection
Before programming anything (including the simulator), first read through the tasks below and make sure you understand them. Then, work with your lab partner(s) (or complete separately and then compare answers) to fill in the Lab Pre-flection worksheet linked in the Lab 1 Pre-flection assignment on Canvas and submit this through GradeScope (only one submission needed per group).

Subtask 3.A – Navigating a Known Distance:

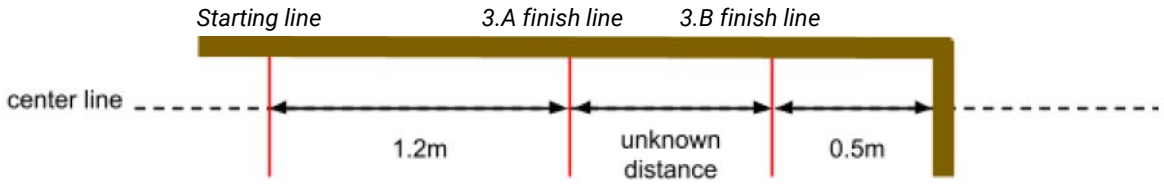
1. Place your robot behind a starting line with a specific measuring point located above the center of the start line.
2. When instructed by the TA or Instructor, you will press the center dark gray button to start your program.
    a. When working with the Webots simulator, you can do this by pressing the “Enter” key on your keyboard, which is mapped to the EV3Brick’s Button.CENTER.
3. The robot should move forward in a straight line for 1.2 meters and stop.
4. Immediately after stopping, the robot should beep.
5. The TA or Instructor will then measure the distance from the starting line and deviation from the center line (movement left and right). Points will be awarded based on accuracy.
6. The robot should remain stopped until the center button is pressed again.

Subtask 3.B – Navigating an Unknown Distance and Responding to Obstacles:

1. After completing the first objective, when instructed by the TA or Instructor, you will press the center dark gray button again.
    a. When working with the Webots simulator, you can do this by pressing the “Enter” key on your keyboard, which is mapped to the EV3Brick’s Button.CENTER.
2. Your robot should resume moving forward until it contacts a large flat surface (like a wall or large object) that will be located some unknown distance in front of the robot.
    a. You can assume all walls or objects will be at least 17 cm high and can be detected by the ultrasonic sensor or touch sensor.
3. After bumping into the surface, the robot should begin backing up (i.e., moving in reverse).
4. The robot should reverse until it is 50 cm from the surface (measured from the measuring point on the robot).

5. The robot should stop and beep and the TA or Instructor will measure the distance from the surface and deviation from the center line. Points will be awarded based on accuracy.

A visual representation of the workspace looking down from the ceiling:



As you can see in this diagram, there will be a wall to the left of the robot (based on its initial pose at the starting line) that runs parallel to the robot's direction of movement. You may want to use this wall as feedback to your control system to ensure that the robot is moving straight (i.e., minimizing left/right deviations). Remember that both this wall and the wall at the end of the course will be at least 17 cm high and can be detected by the ultrasonic sensor or touch sensor. You can also refer to the Webots lab1.wbt world file that provides an example environment matching this definition. As you are testing your code, you may want to edit this world file (e.g., to change the distances between the 3.A finish line and the 3.B finish line + wall); please see the instructions in the GitHub repository README.md file for information on this (and remember what you learned in Lab 0 about editing Webots environments).

Reflection

After finishing the lab, work with your lab partner to fill in the Lab Reflection worksheet linked in the Lab 1 Reflection assignment on Canvas and submit this through GradeScope (only one submission needed per group).

Code

To complete this lab, you will use a subset of the EV3 MicroPython API. You may only use the following modules:
ev3brick
ev3devices
parameters
tools
media

You may *not* import, use, or copy the "robotics" module, or any of its included classes or functions, in your program.

A robot relying on a module not allowed as stated above will be disqualified. Also, any robot using Wi-Fi or Bluetooth communications in any way will be disqualified.

Make sure to commit and push your final code to GitHub. You must also submit a .zip file of your Python source code files on GradeScope. The .zip file should include all the Python source code files you wrote (which should have a .py file extension). In the comments at the top of the Python source file containing your main .py file, please include the names and PID's of all team members. Please make it obvious somehow which .py file is used for your simulator and which is used for your physical robot (i.e., include this information in the file name or in the comments).

You are free to discuss concepts in the course with other students or AI tools (e.g., ChatGPT) in natural language, but you should not share any programming code with others and you must write all the robot's programming code yourself. If you access any sources other than the textbook or course material posted on Canvas, you must cite them in comments at the top of your code. You must fully understand your code and be able to reproduce the algorithmic approach without references if asked. The Honor Code is in effect for these policies.

Grading Rubric

- **(5 Points) Pre-flection:**
  - Effort: 5 points. Points will be awarded for submitting the Pre-flection on time and for making a reasonable effort to answer each question.
- **(45 Points) Simulator:**
  - Initial code: 5 points. Points will be awarded if you upload your Webots simulator code to GitHub and demonstrate to the TA that appears to have a reasonable effort in (e.g., at minimum the robot moves and shows some attempt towards accomplishing Task 3.A) by the simulator deadline.
  - Correct code: 40 points. Points will be awarded if you upload your code which successfully accomplishes the tasks in the simulator by the full lab deadline. The division of points across simulator tasks 3.A and 3.B is identical to that described below for the real robot attempt.
- **(45 Points) Real Robot Attempt:**
  - Effort: 5 points. You will receive these points by:
    - Having the EV3 MicroPython successfully installed: 1 point
    - Showing up to the lab on time: 1 point
    - Having your robot built and ready to attempt the course: 1 point
    - Pointing out your robot's measuring point according to the definition given in this document: 1 point
  - Task 3.A: 20 points, distributed as follows:
    - 10 points will be awarded based on correct distance traveled
    - 10 points for minimizing left/right deviation
  - Task 3.B: 20 points, distributed as follows:
    - 10 points will be awarded for bumping the wall and reversing
    - 5 points for achieving the correct final distance from the wall
    - 5 points for minimizing left/right deviation from the center line
- **(5 Points) Reflection:**
  - Effort: 5 points. Points will be awarded for submitting the Reflection on time and for making a reasonable effort to answer each question.

All distance measurements with errors <10mm are guaranteed to receive full credit.

Good Luck and Have Fun!